LABORATORY WORK 7

Indexes

Database Management Systems

Task 1 — Index on actual departure

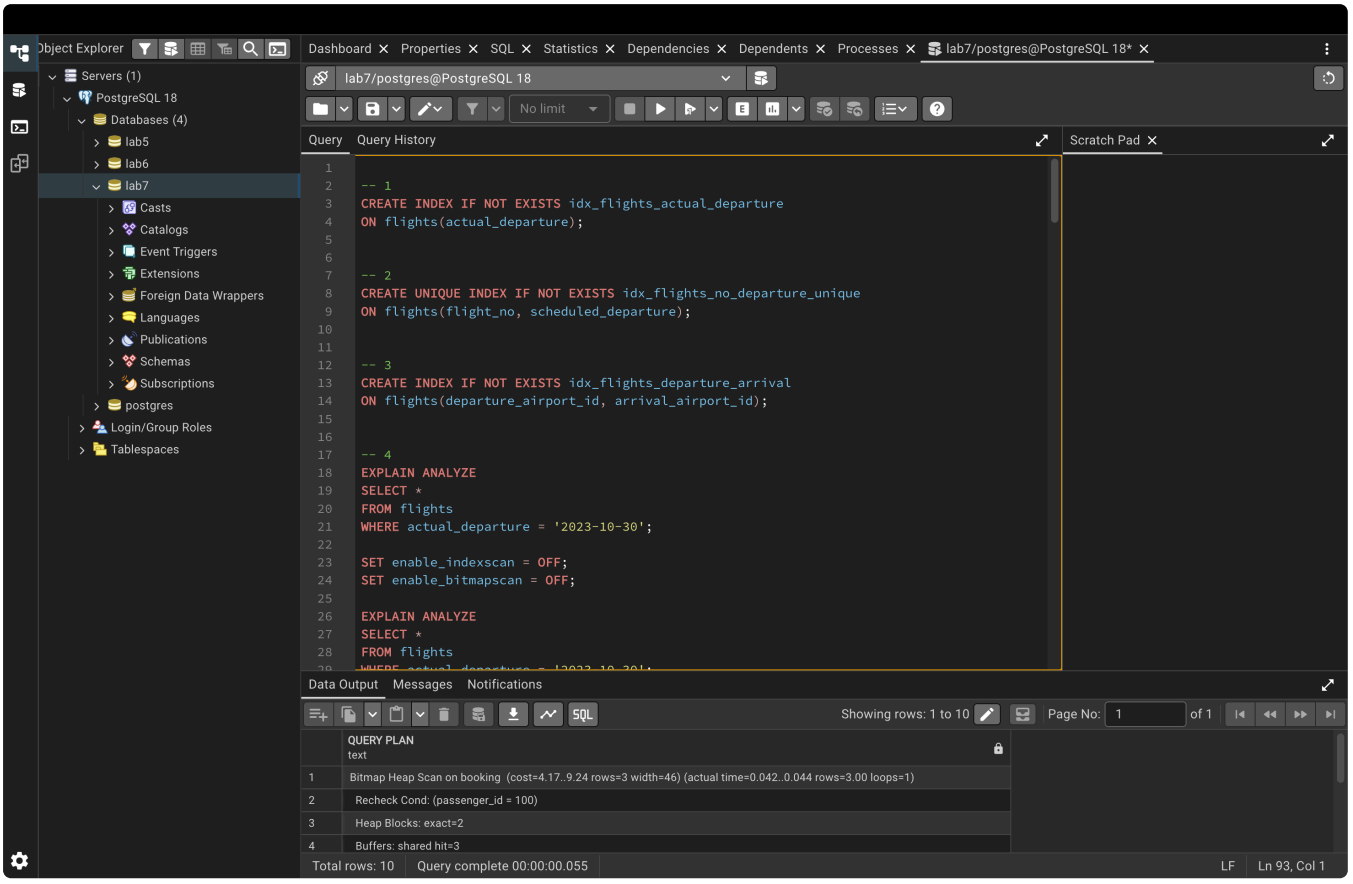
Explanation: A B-tree index is created on the actual_departure column of the flights table to speed up searches by actual departure date.

SQL Query

```
CREATE INDEX IF NOT EXISTS idx_flights_actual_departure
ON flights(actual_departure);
```

Result: The index was created successfully.

Screenshot — Task 1



Task 2 — Unique index on flight number and scheduled departure

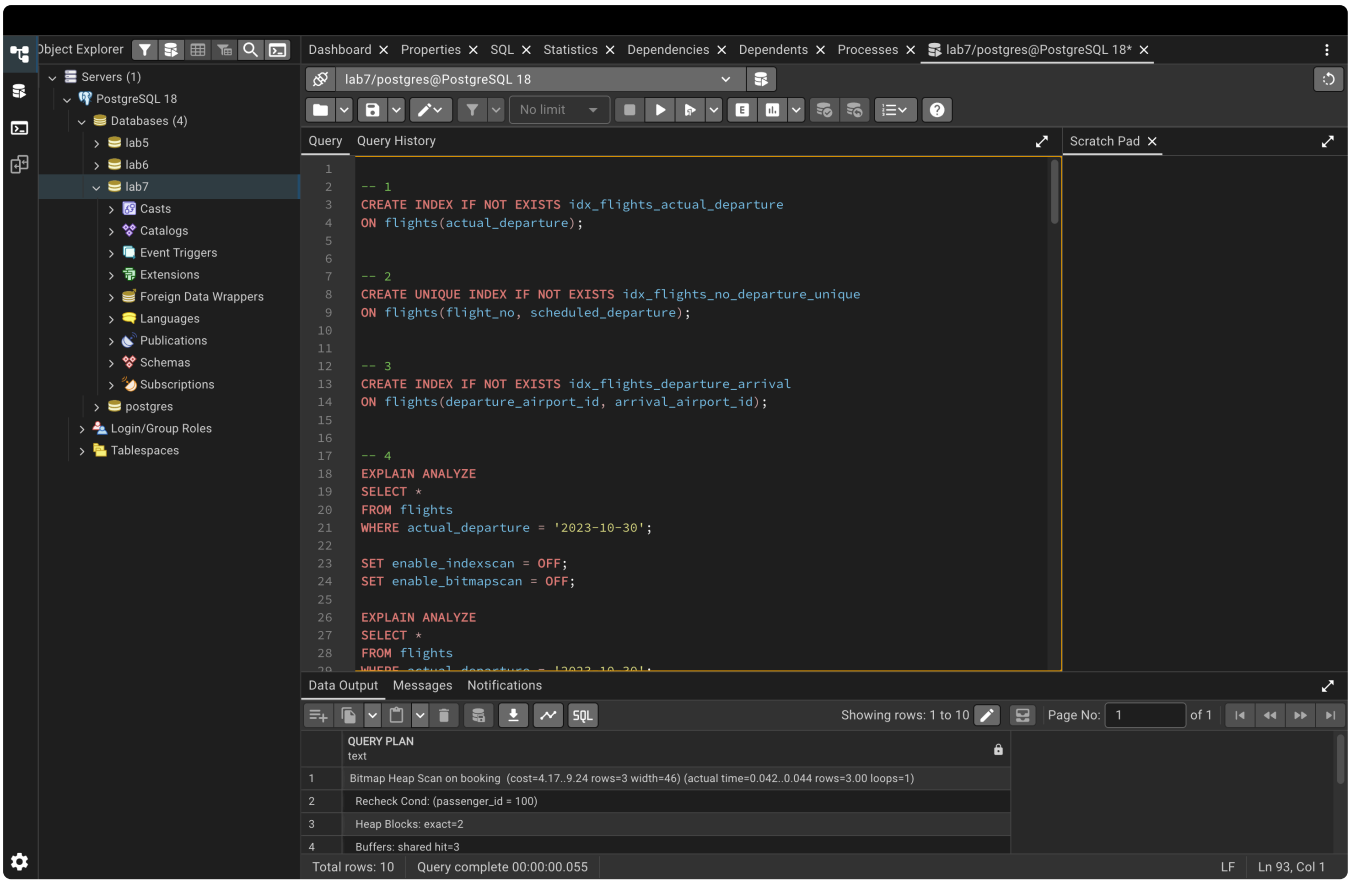
Explanation: A unique composite index guarantees that the same flight number cannot be scheduled twice for the same departure time.

SQL Query

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_flights_no_departure_unique
ON flights(flight_no, scheduled_departure);
```

Result: The unique index was created successfully, so the combination (flight_no, scheduled_departure) is now unique.

Screenshot — Task 2



Task 3 — Composite index on departure and arrival airports

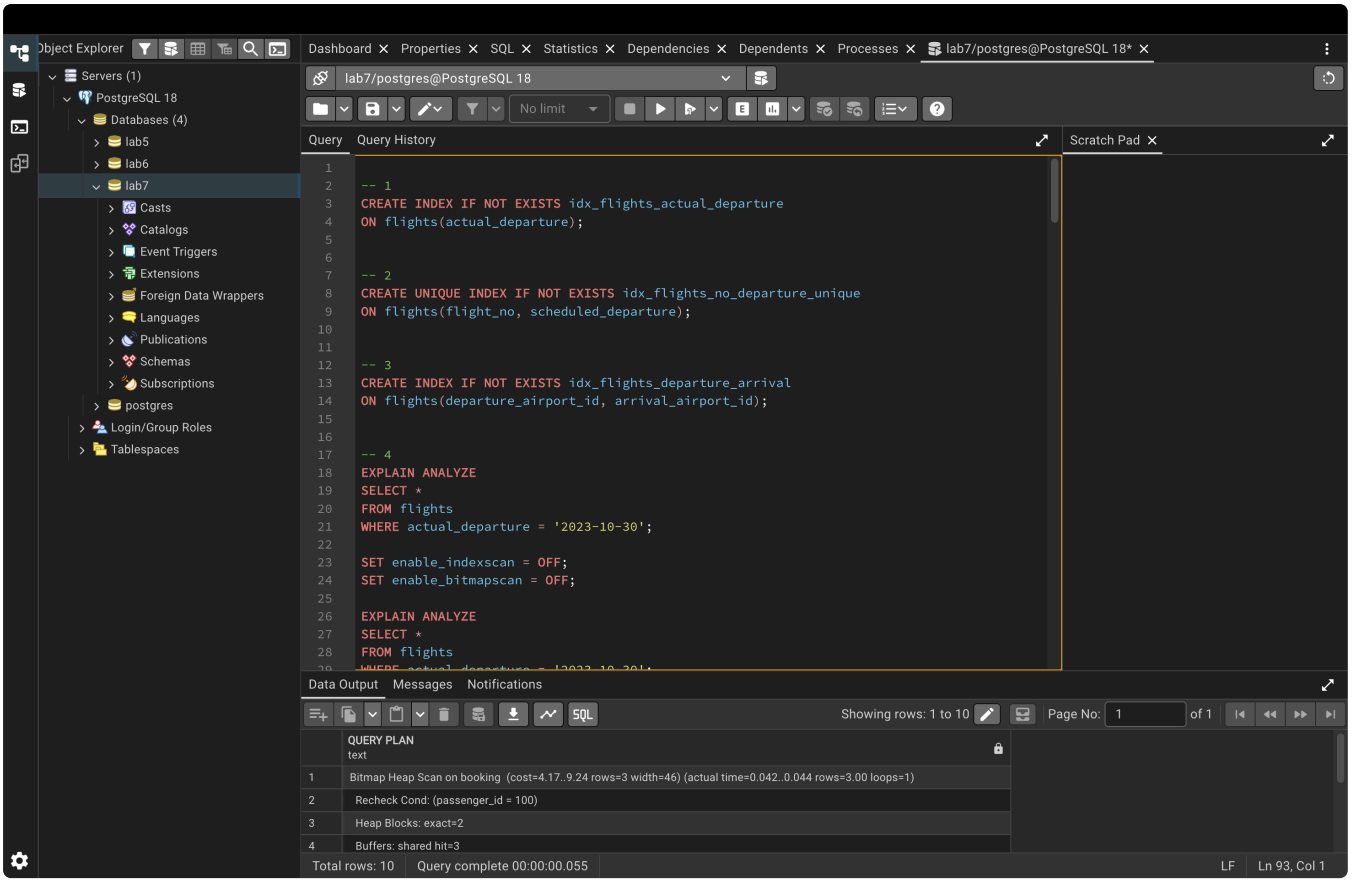
Explanation: A composite index on departure_airport_id and arrival_airport_id is created for searching flights by route.

SQL Query

```
CREATE INDEX IF NOT EXISTS idx_flights_departure_arrival
ON flights(departure_airport_id, arrival_airport_id);
```

Result: The index was created successfully.

Screenshot — Task 3



Task 4 — Query plan with and without the index

Explanation: The query plan for a search by actual_departure is analyzed with EXPLAIN ANALYZE. Then index scan and bitmap scan are disabled to force the planner to use a sequential scan, and the two plans are compared. After the test, both settings are turned back on.

SQL Query

```
EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE actual_departure = '2023-10-30';

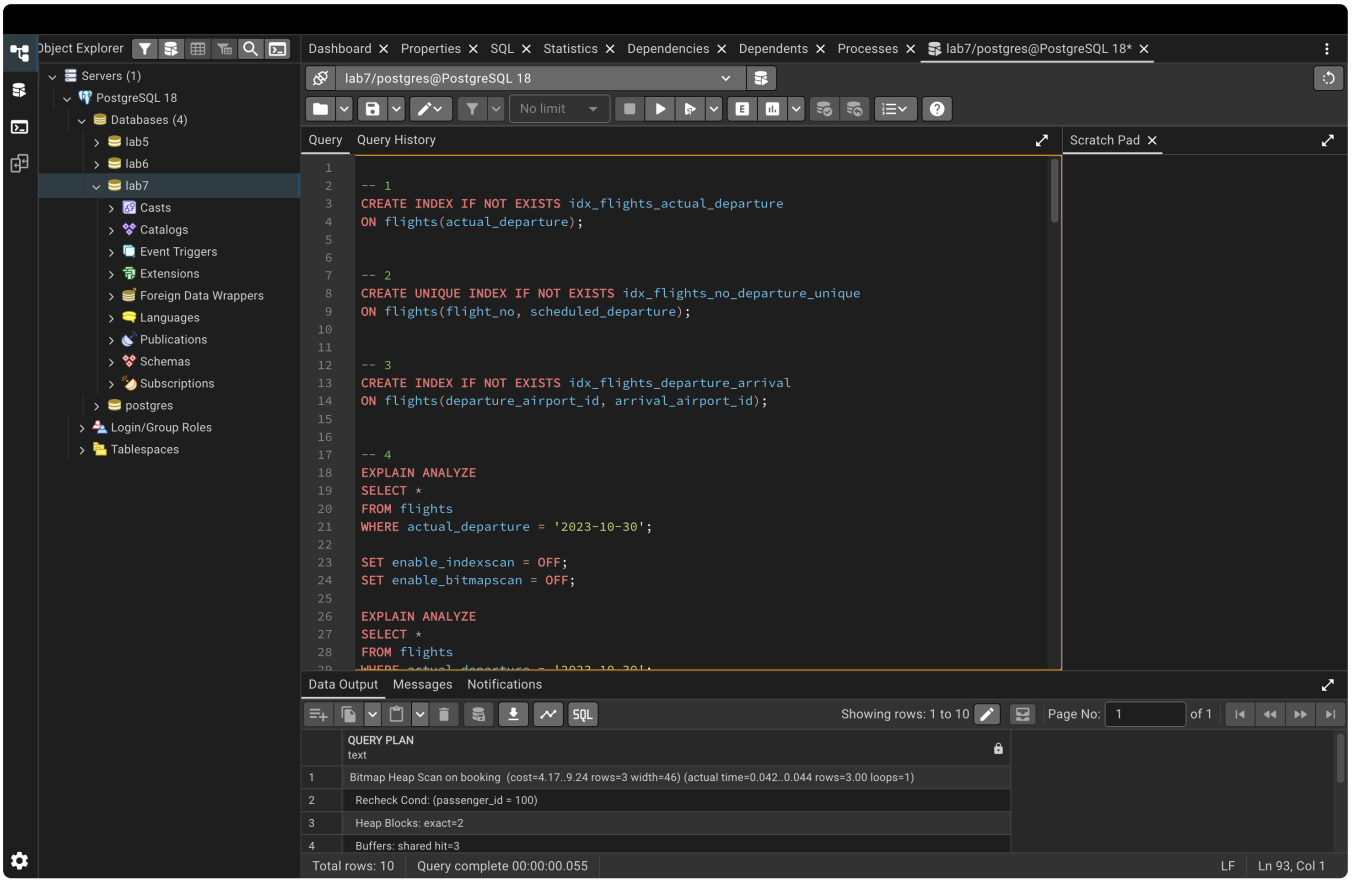
SET enable_indexscan = OFF;
SET enable_bitmapscan = OFF;

EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE actual_departure = '2023-10-30';

SET enable_indexscan = ON;
SET enable_bitmapscan = ON;
```

Result: With the index enabled, the planner can use idx_flights_actual_departure. With index and bitmap scans disabled, PostgreSQL has to read the whole table (Seq Scan), which is slower on large tables.

Screenshot — Task 4



Task 5 — Using the composite index

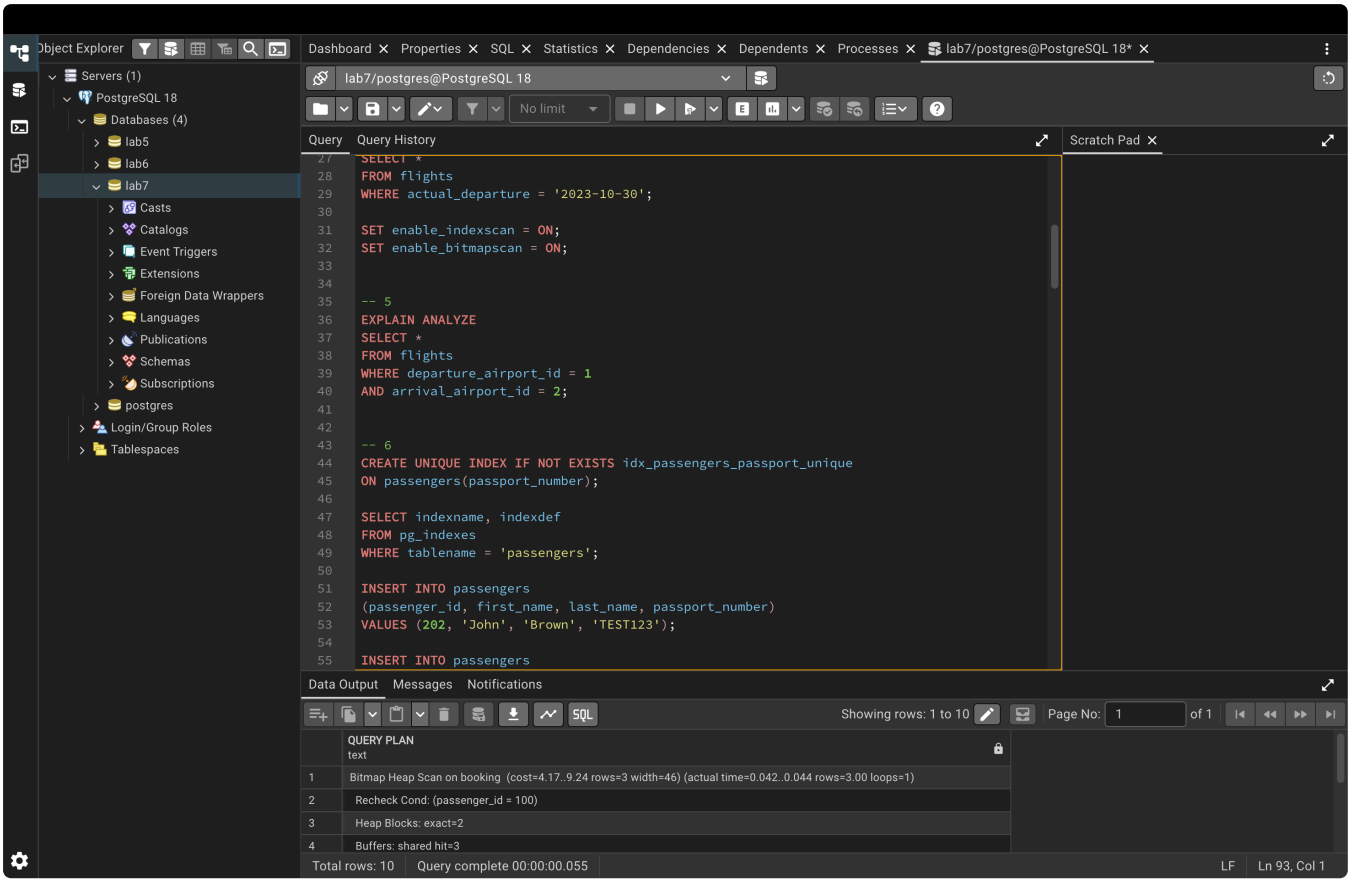
Explanation: A query filtering by both columns of the composite index (departure_airport_id and arrival_airport_id) is analyzed.

SQL Query

```
EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE departure_airport_id = 1
AND arrival_airport_id = 2;
```

Result: The condition matches the columns of idx_flights_departure_arrival in the correct order, so the planner can use this index instead of scanning the whole table.

Screenshot — Task 5



Task 6 — Unique index on passport number

Explanation: A unique index is created on passport_number so that two passengers cannot have the same passport. The indexes of the passengers table are listed through pg_indexes, and then a duplicate insert is tested.

SQL Query

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_passengers_passport_unique
ON passengers(passport_number);

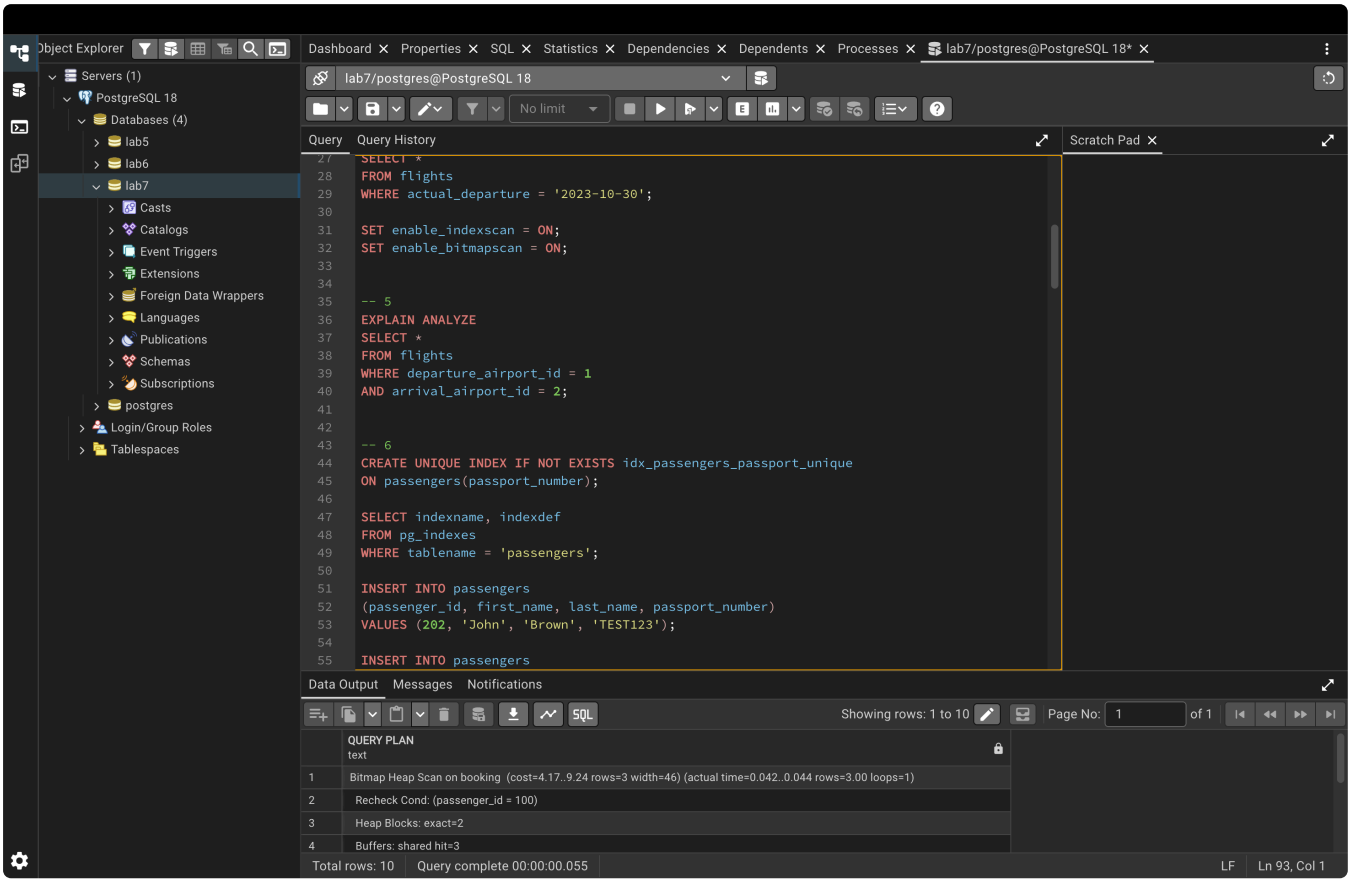
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';

INSERT INTO passengers
(passenger_id, first_name, last_name, passport_number)
VALUES (202, 'John', 'Brown', 'TEST123');

INSERT INTO passengers
(passenger_id, first_name, last_name, passport_number)
VALUES (202, 'John', 'Brown', 'TEST123');
```

Result: The first INSERT is executed successfully. The second INSERT with the same data is rejected with a duplicate key (unique violation) error, which confirms that the unique index works.

Screenshot — Task 6



Task 7 — Composite index on passenger data

Explanation: A composite index on first_name, last_name, date_of_birth and country_of_citizenship is created. Then a query filtering only by country_of_citizenship and date_of_birth is analyzed.

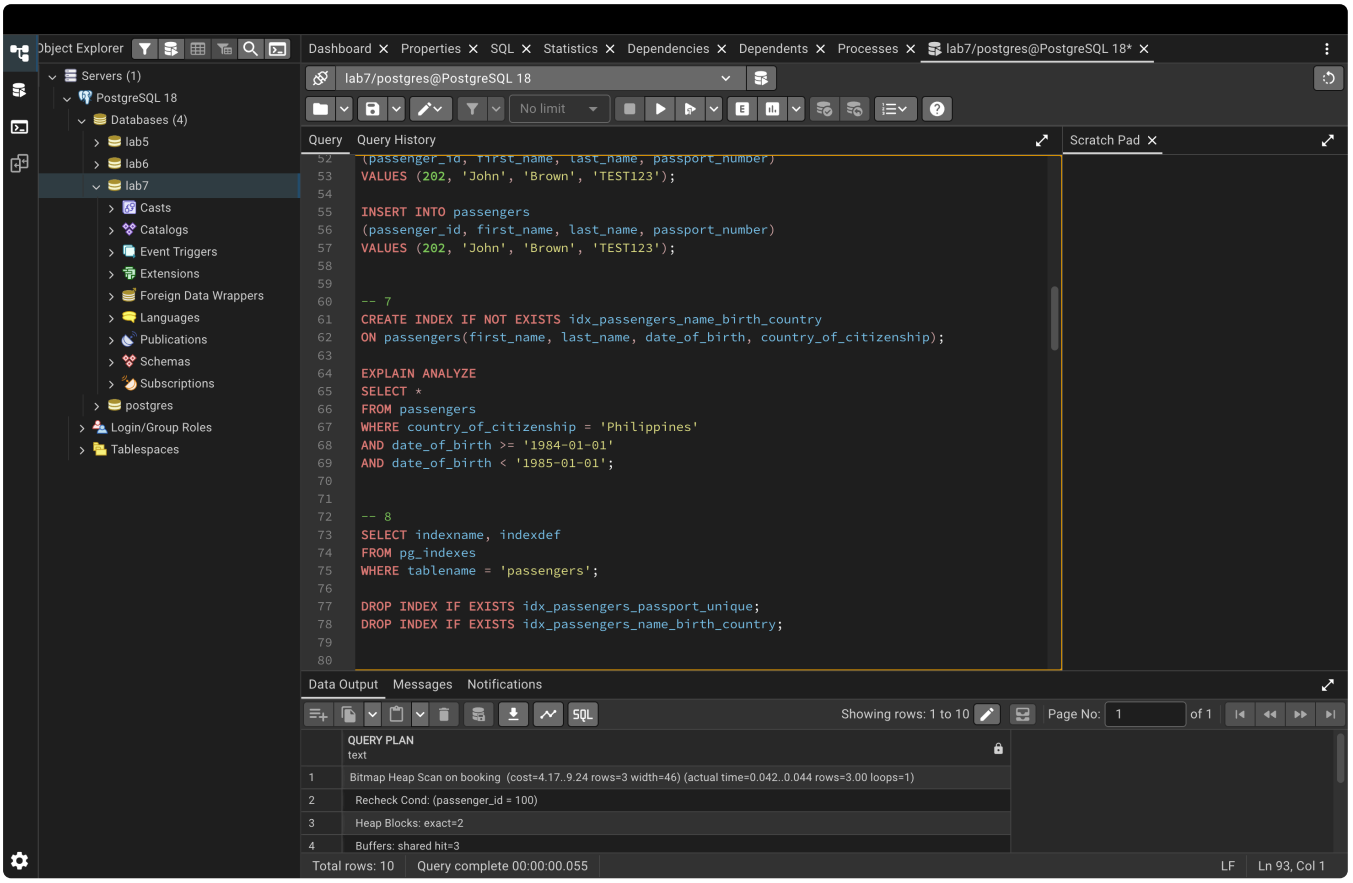
SQL Query

```
CREATE INDEX IF NOT EXISTS idx_passengers_name_birth_country
ON passengers(first_name, last_name, date_of_birth, country_of_citizenship);

EXPLAIN ANALYZE
SELECT *
FROM passengers
WHERE country_of_citizenship = 'Philippines'
AND date_of_birth >= '1984-01-01'
AND date_of_birth < '1985-01-01';
```

Result: The query does not use the first columns of the index (first_name, last_name), so this composite index is not efficient for it. The order of columns in a composite index matters: the index is useful when the leading columns are present in the condition.

Screenshot — Task 7



Task 8 — Viewing and dropping indexes

Explanation: The indexes of the passengers table are displayed from pg_indexes, and the two indexes created earlier are dropped.

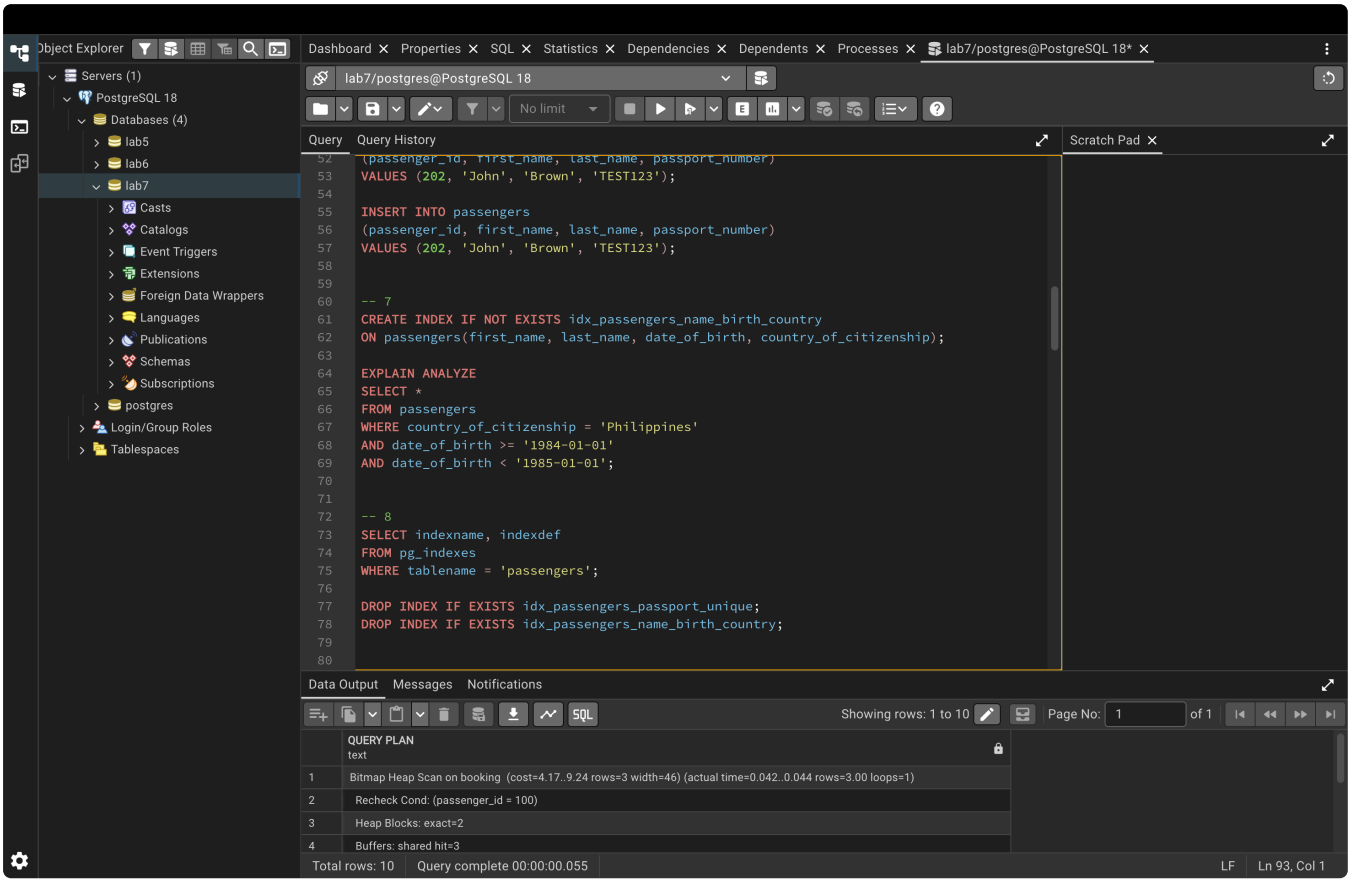
SQL Query

```
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';

DROP INDEX IF EXISTS idx_passengers_passport_unique;
DROP INDEX IF EXISTS idx_passengers_name_birth_country;
```

Result: The indexes were listed and then removed successfully. Unused indexes should be dropped because they slow down INSERT, UPDATE and DELETE operations and take disk space.

Screenshot — Task 8



Task 9 — Index on foreign key booking.passenger_id

Explanation: The same search by passenger_id is executed before and after creating an index on the foreign key column of the booking table.

SQL Query

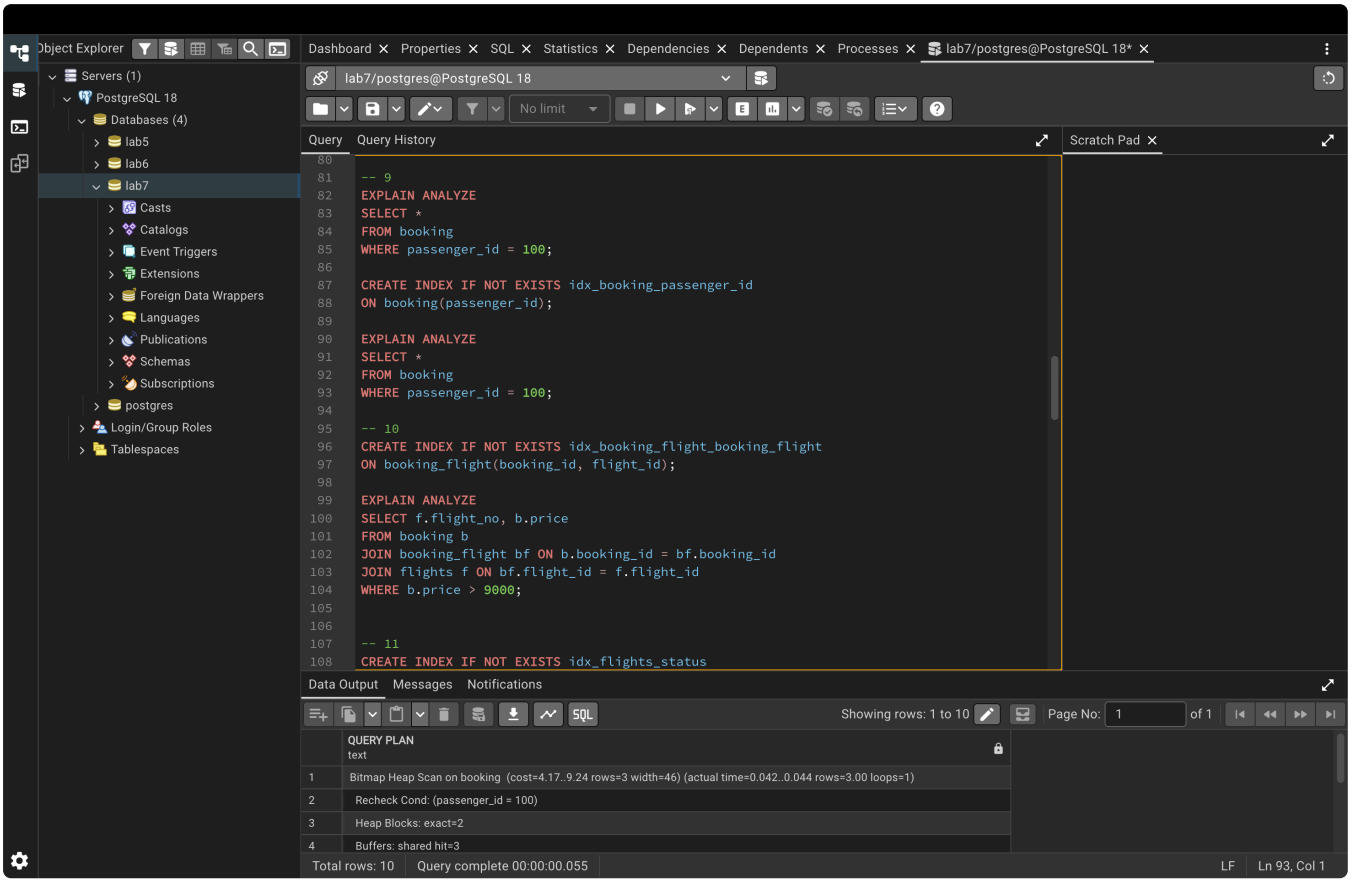
```
EXPLAIN ANALYZE
SELECT *
FROM booking
WHERE passenger_id = 100;

CREATE INDEX IF NOT EXISTS idx_booking_passenger_id
ON booking(passenger_id);

EXPLAIN ANALYZE
SELECT *
FROM booking
WHERE passenger_id = 100;
```

Result: After the index was created, the plan shows a Bitmap Heap Scan on booking with Recheck Cond: (passenger_id = 100), cost=4.17..9.24, 3 rows, actual time about 0.042..0.044 ms. The planner uses the index instead of reading the whole table.

Screenshot — Task 9



Task 10 — Index for JOIN

Explanation: A composite index on booking_flight(booking_id, flight_id) is created to speed up joining booking, booking_flight and flights.

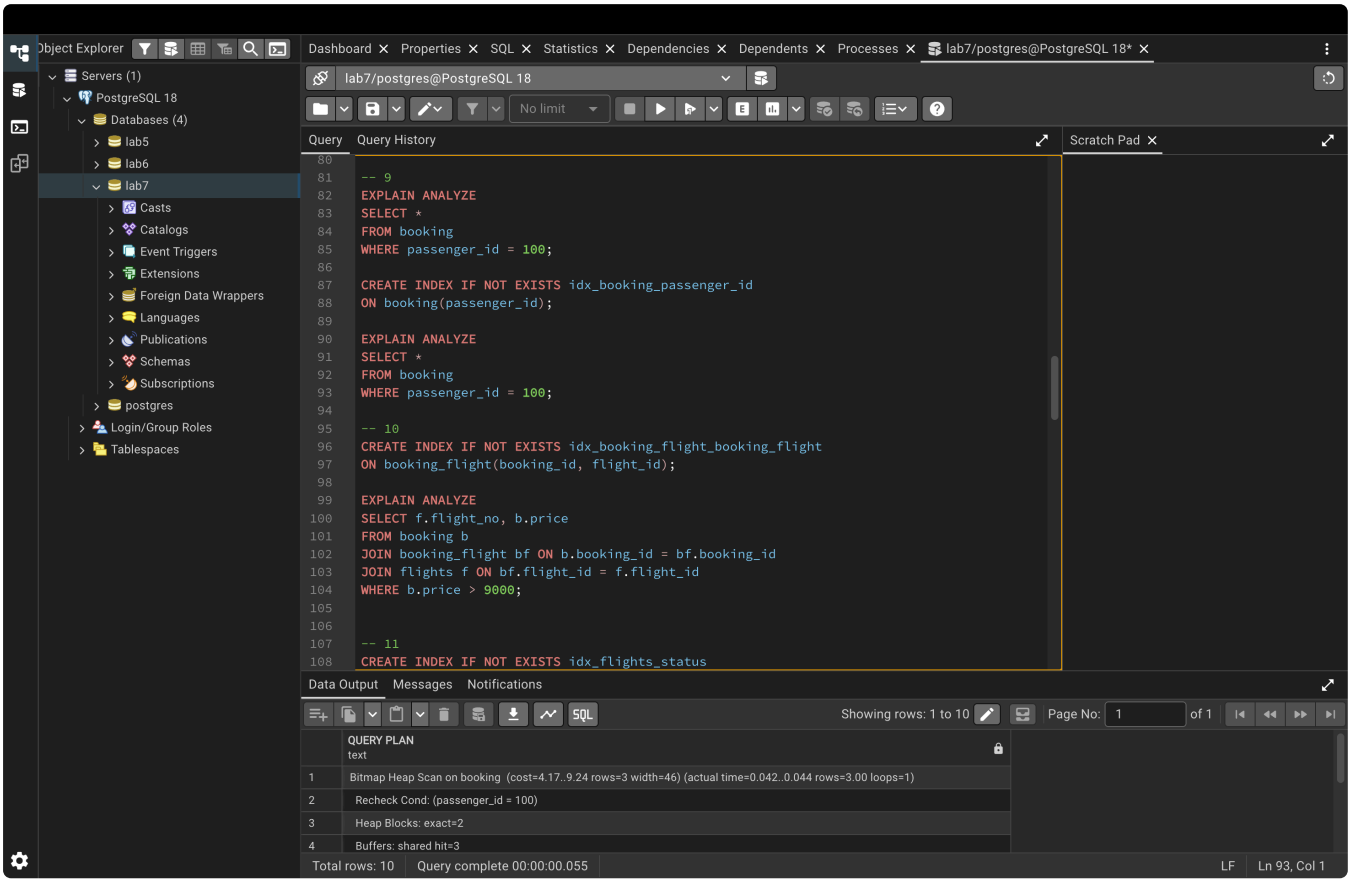
SQL Query

```
CREATE INDEX IF NOT EXISTS idx_booking_flight_booking_flight
ON booking_flight(booking_id, flight_id);

EXPLAIN ANALYZE
SELECT f.flight_no, b.price
FROM booking b
JOIN booking_flight bf ON b.booking_id = bf.booking_id
JOIN flights f ON bf.flight_id = f.flight_id
WHERE b.price > 9000;
```

Result: The index covers both join columns of booking_flight, so the planner can use it when joining with booking and flights. The plan shows how the join is executed after filtering by price.

Screenshot — Task 10



Task 11 — Index on a column with few distinct values

Explanation: An index on the status column of flights is created, and two searches with different statuses are compared.

SQL Query

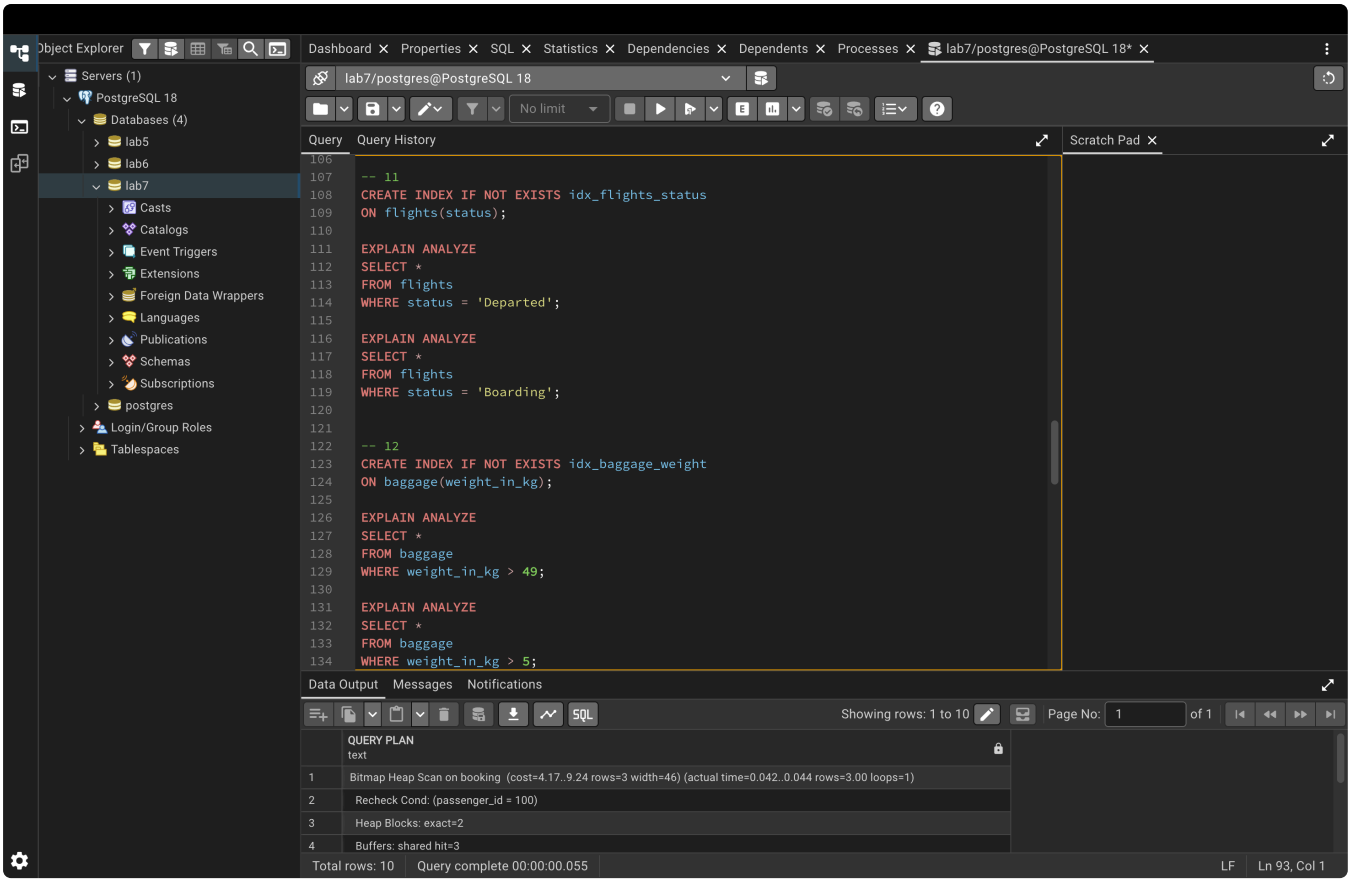
```
CREATE INDEX IF NOT EXISTS idx_flights_status
ON flights(status);

EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE status = 'Departed';

EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE status = 'Boarding';
```

Result: The status column has only a few distinct values. For a frequent value ('Departed') the planner may prefer a sequential scan because a large part of the table must be read anyway. For a rare value ('Boarding') the index is more useful. This shows that index efficiency depends on selectivity.

Screenshot — Task 11



Task 12 — Selectivity of a condition

Explanation: An index on baggage.weight_in_kg is created and two range queries are compared: a very selective one (weight > 49) and a non-selective one (weight > 5).

SQL Query

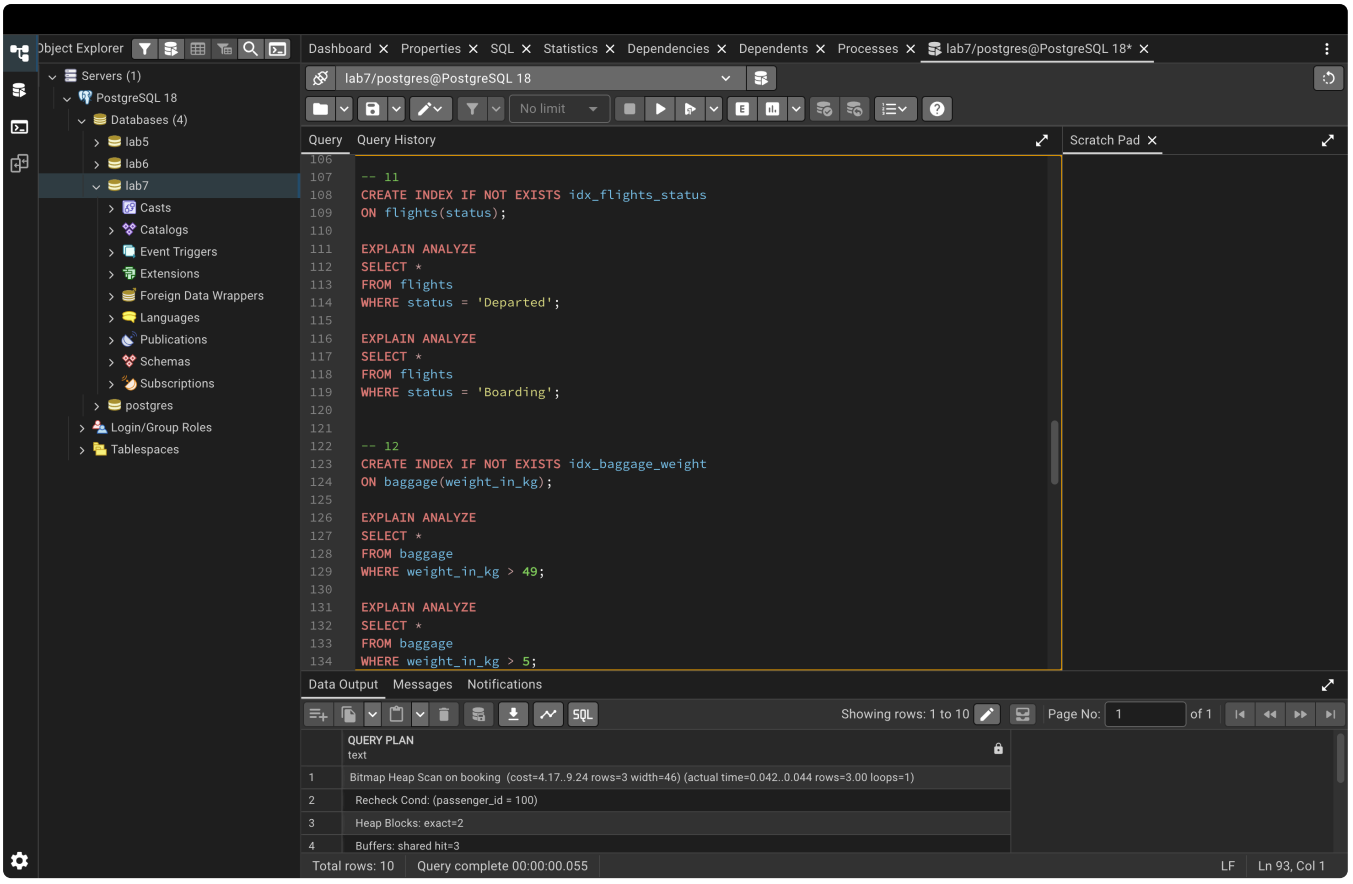
```
CREATE INDEX IF NOT EXISTS idx_baggage_weight
ON baggage(weight_in_kg);

EXPLAIN ANALYZE
SELECT *
FROM baggage
WHERE weight_in_kg > 49;

EXPLAIN ANALYZE
SELECT *
FROM baggage
WHERE weight_in_kg > 5;
```

Result: The condition weight_in_kg > 49 returns few rows, so the index is effective. The condition weight_in_kg > 5 returns most of the table, so the planner usually chooses a sequential scan.

Screenshot — Task 12



Task 13 — Index and LIKE patterns

Explanation: An index on airport_name is created and two LIKE searches are compared: a search by prefix and a search by suffix.

SQL Query

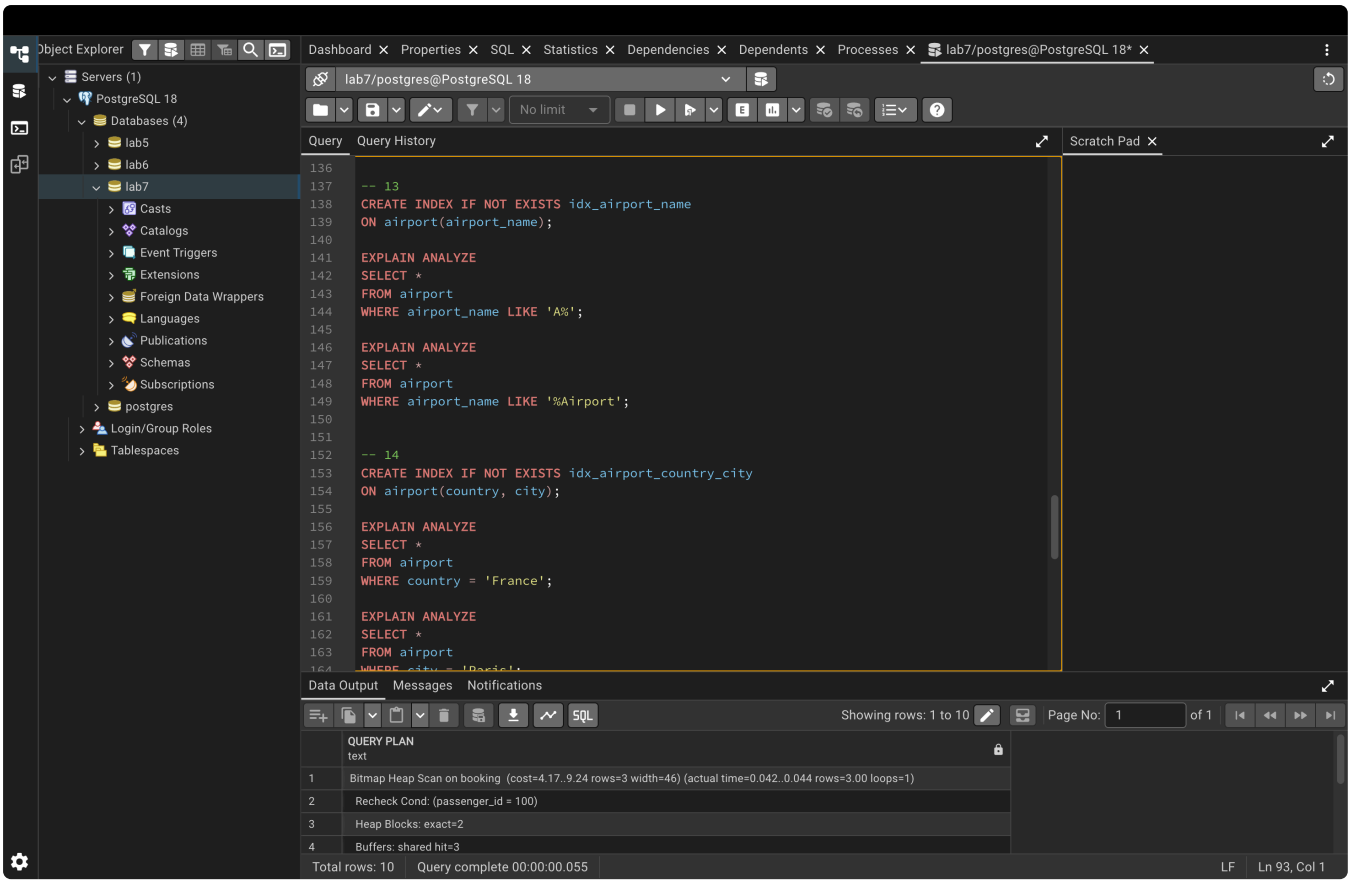
```
CREATE INDEX IF NOT EXISTS idx_airport_name
ON airport(airport_name);

EXPLAIN ANALYZE
SELECT *
FROM airport
WHERE airport_name LIKE 'A%';

EXPLAIN ANALYZE
SELECT *
FROM airport
WHERE airport_name LIKE '%Airport';
```

Result: A pattern with a fixed beginning ('A%') can use a B-tree index (depending on the database collation). A pattern that starts with % ('%Airport') cannot use a regular B-tree index, so a sequential scan is used.

Screenshot — Task 13



Task 14 — Composite index and column order

Explanation: A composite index on airport(country, city) is created. Three queries are analyzed: by country only, by city only, and by both columns.

SQL Query

```
CREATE INDEX IF NOT EXISTS idx_airport_country_city
ON airport(country, city);

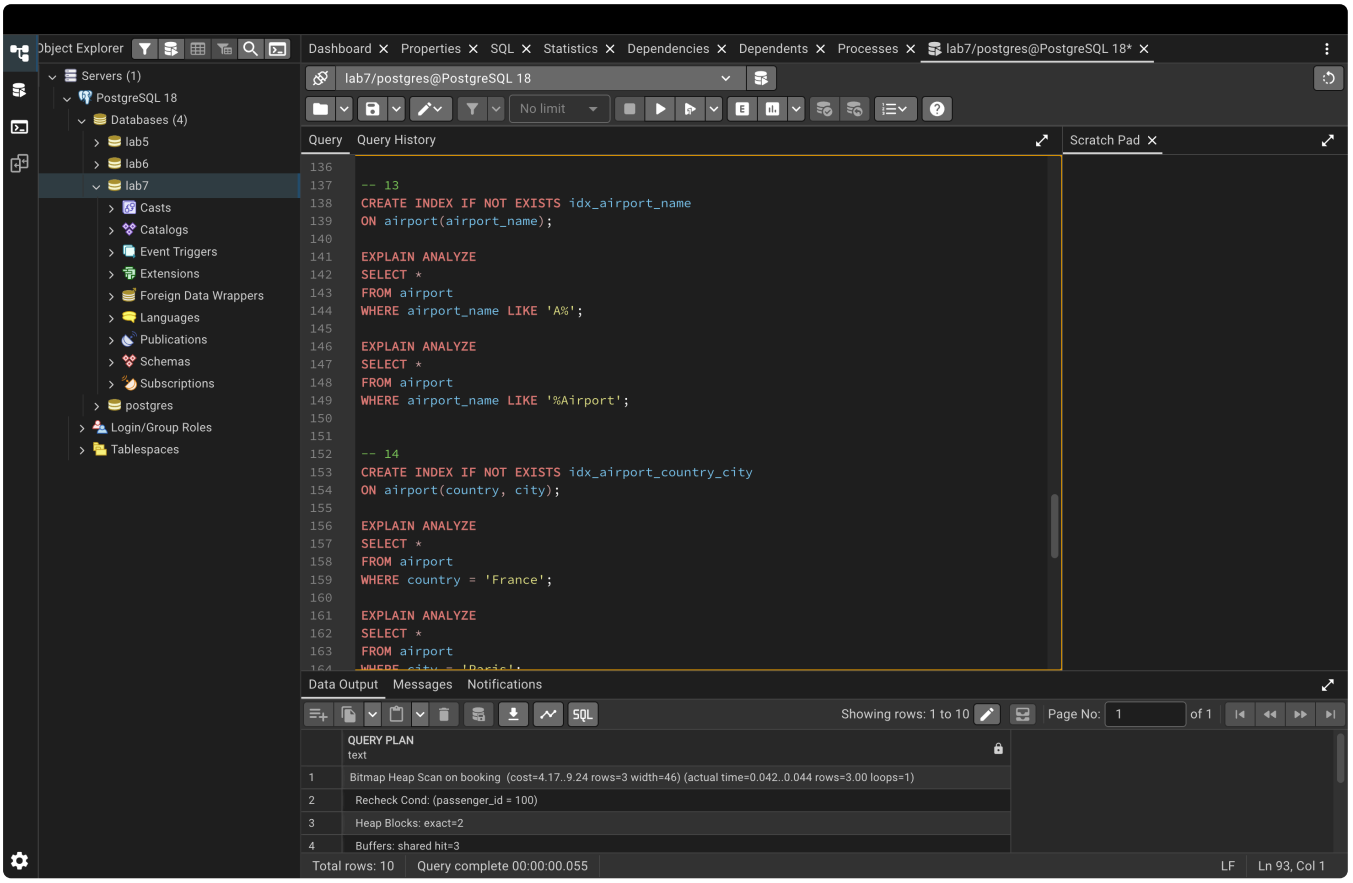
EXPLAIN ANALYZE
SELECT *
FROM airport
WHERE country = 'France';

EXPLAIN ANALYZE
SELECT *
FROM airport
WHERE city = 'Paris';

EXPLAIN ANALYZE
SELECT *
FROM airport
WHERE country = 'France'
AND city = 'Paris';
```

Result: The index works best for queries on country and on both columns, because country is the leading column. The query only by city does not use the leading column, so the index is not efficient for it.

Screenshot — Task 14



Task 15 — Functional index

Explanation: An index on LOWER(last_name) is created for case-insensitive search. Then a search with LOWER() and a regular search by last_name are compared.

SQL Query

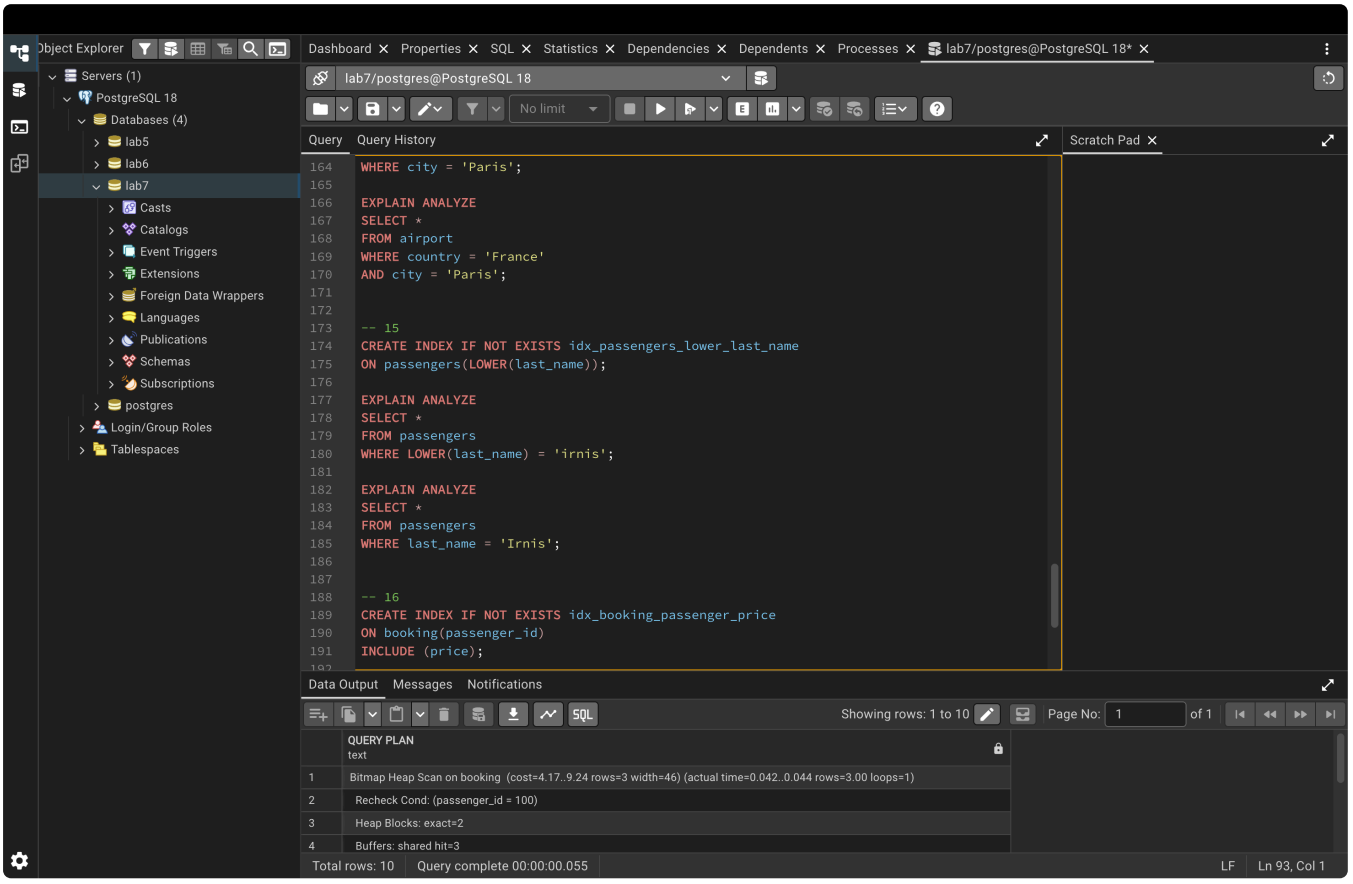
```
CREATE INDEX IF NOT EXISTS idx_passengers_lower_last_name
ON passengers(LOWER(last_name));

EXPLAIN ANALYZE
SELECT *
FROM passengers
WHERE LOWER(last_name) = 'irnis';

EXPLAIN ANALYZE
SELECT *
FROM passengers
WHERE last_name = 'Irnis';
```

Result: The query with LOWER(last_name) matches the expression of the functional index and can use it. The query with last_name = 'Irnis' does not match the indexed expression, so this index is not used for it.

Screenshot — Task 15



Task 16 — Covering index (INCLUDE)

Explanation: An index on booking(passenger_id) with the included column price is created. The query selects only price by passenger_id, so all needed data can be taken from the index.

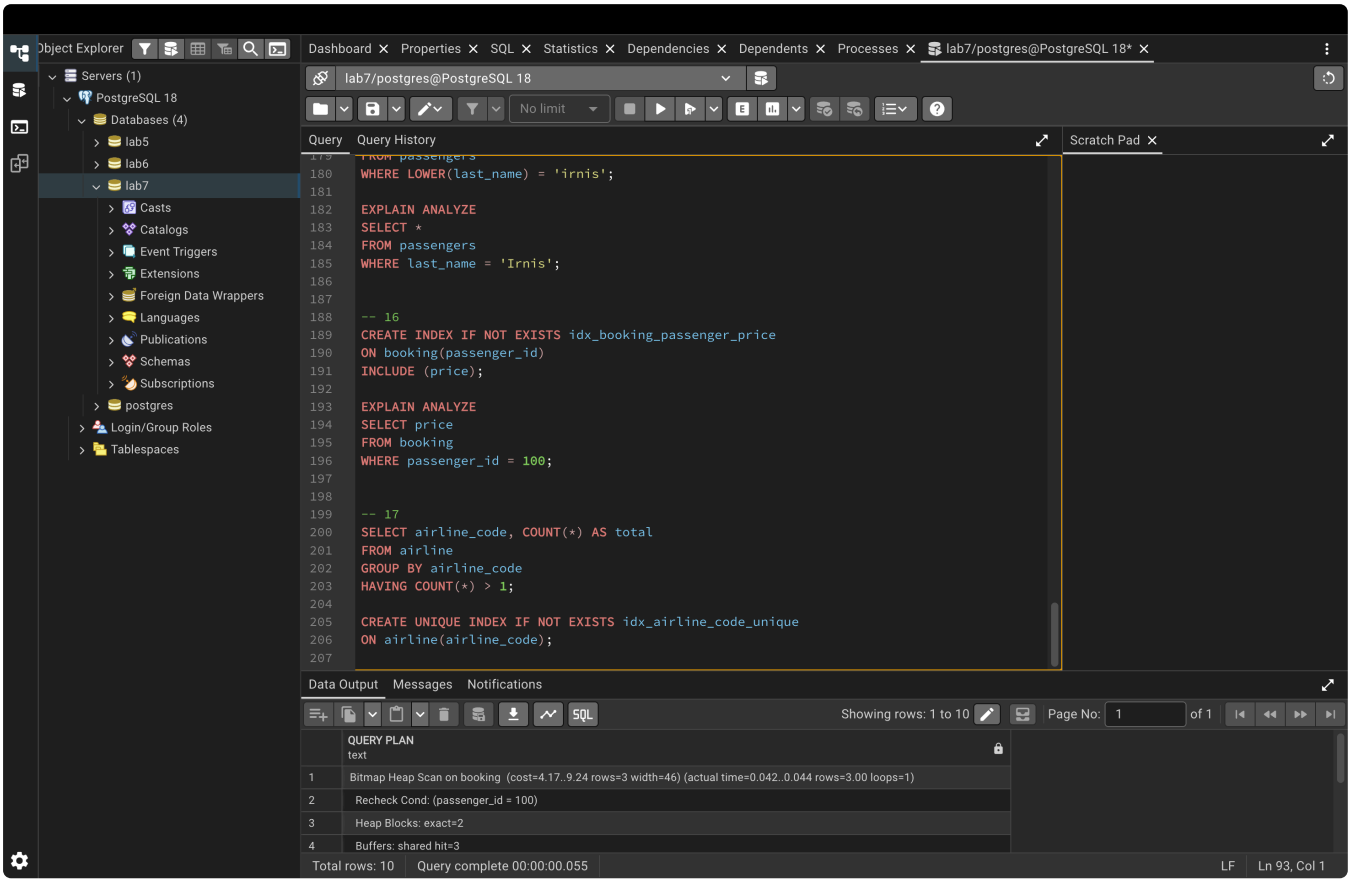
SQL Query

```
CREATE INDEX IF NOT EXISTS idx_booking_passenger_price
ON booking(passenger_id)
INCLUDE (price);

EXPLAIN ANALYZE
SELECT price
FROM booking
WHERE passenger_id = 100;
```

Result: Since both passenger_id and price are stored in the index, PostgreSQL can use an Index Only Scan and does not need to read the table rows (it depends on the visibility map, so VACUUM may be needed).

Screenshot — Task 16



Task 17 — Unique index on airline code

Explanation: Before creating a unique index, the data is checked for duplicates of airline_code. If there are no duplicates, the unique index is created.

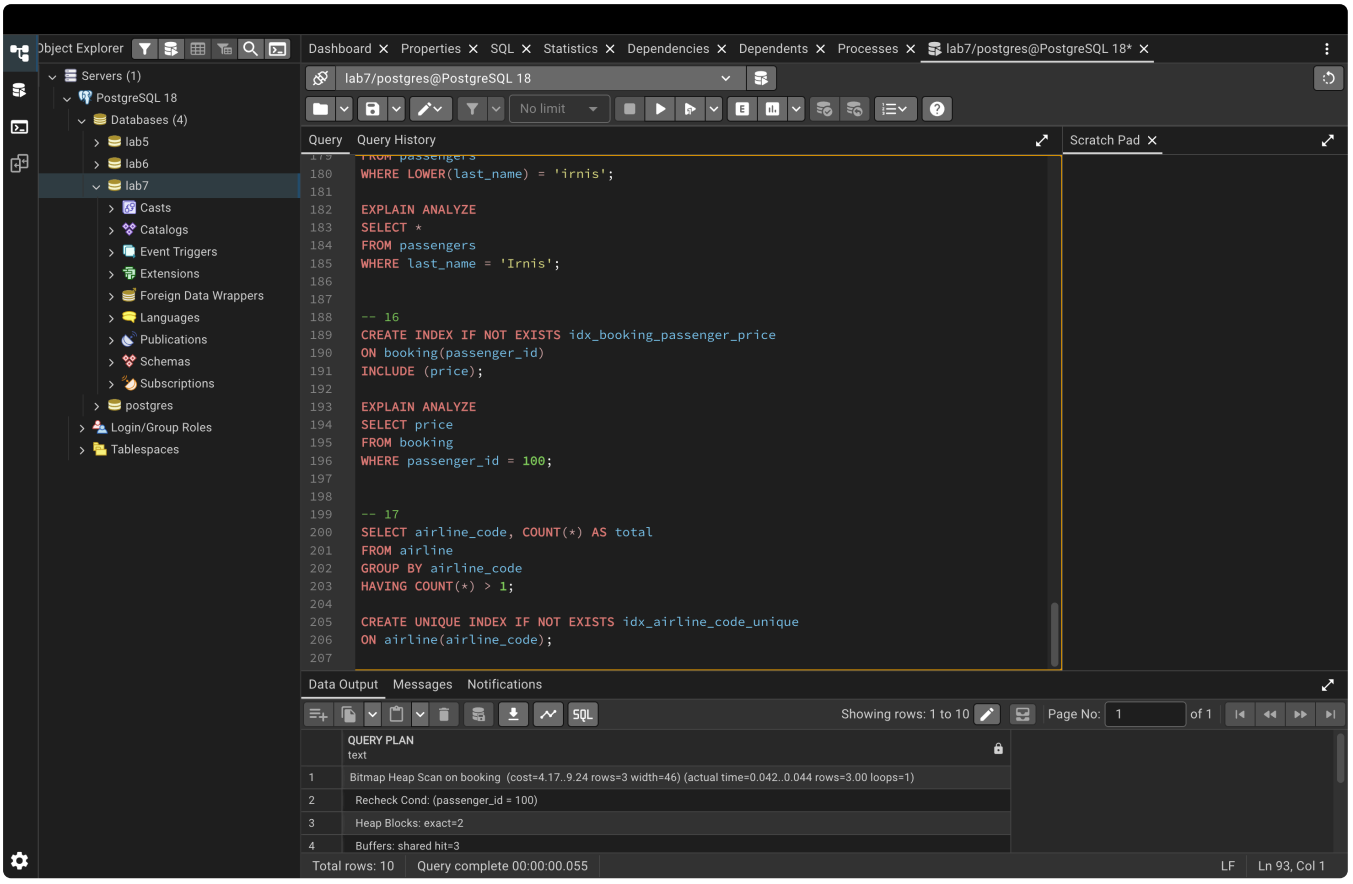
SQL Query

```
SELECT airline_code, COUNT(*) AS total
FROM airline
GROUP BY airline_code
HAVING COUNT(*) > 1;

CREATE UNIQUE INDEX IF NOT EXISTS idx_airline_code_unique
ON airline(airline_code);
```

Result: The check query shows whether duplicate codes exist. If it returns no rows, the unique index is created successfully and prevents duplicate airline codes in the future.

Screenshot — Task 17



Conclusion

During Laboratory Work 7, different types of indexes in PostgreSQL were created and tested: simple, composite, unique, functional and covering (INCLUDE) indexes. EXPLAIN ANALYZE was used to compare query plans and see when the planner chooses Seq Scan, Index Scan, Bitmap Heap Scan or Index Only Scan. It was shown that the efficiency of an index depends on selectivity of the condition, on the column order in composite indexes, on the form of the LIKE pattern and on matching the indexed expression. Unique indexes were used to guarantee data integrity, and unused indexes were dropped with DROP INDEX because they slow down data modification.